

CAP1002:COMPUTATIONAL THINKING WITH PYTHON

L:3 T:0 P:2 Credits:4

Course Outcomes: Through this course students should be able to

CO1 :: identify the fundamental concepts of computational thinking, algorithms, and Python programming.

CO2 :: explain problem-solving techniques, Python programming constructs, and commonly used data structures for computational applications.

CO3 :: interpret computational problems by applying decomposition, abstraction, and pattern recognition to formulate solution strategies.

CO4 :: develop Python programs using appropriate programming constructs, functions, data structures, and standard libraries to solve basic computational problems.

Unit I

Introduction to Computational Thinking and Python : Introduction to Computational Thinking and Problem Solving, Decomposition, Pattern Recognition, Abstraction, and Algorithm Design, Introduction to Python and Python Development Environment, Variables, Data Types, Input/Output Operations, and Type Conversion, Operators and Expressions

Unit II

Decision Making and Iteration : Conditional Statements (if, if-else, nested if, elif), Iterative Statements (for and while loops), Loop Control Statements (break, continue, and pass), Nested Loops, Problem Solving using Flowcharts and Pseudocode

Unit III

Data Structures and Functions : Strings and String Operations, Lists, Tuples, Sets, and Dictionaries, User-Defined Functions and Function Arguments, Scope of Variables and Return Statements, Modules and Standard Python Libraries

Unit IV

Algorithms and File Handling : Linear Search and Binary Search, Bubble Sort and Selection Sort, Text File Handling (Read, Write, Append), Exception Handling, Program Testing and Debugging

Unit V

Data Processing with Python : Introduction to NumPy Arrays, Introduction to Pandas DataFrames, Reading and Writing CSV Files, Basic Data Manipulation and Summary Statistics, Introduction to Data Visualization using Matplotlib

Unit VI

Python Applications : Introduction to Object-Oriented Programming (Classes and Objects), Using Standard Python Libraries, Solving Real-World Computational Problems, Mini Project Development, Code Documentation and Good Programming Practices

List of Practicals / Experiments:

List of Practicals

- Develop a Python program to perform arithmetic operations, accept user input, and display formatted output using different data types.
- Implement Python programs using conditional statements to solve decision-making problems such as grading systems, leap year determination, and simple menu-driven applications.
- Develop Python programs using for and while loops to generate number patterns, compute mathematical series, and perform repetitive computations.
- Write Python programs to perform operations on strings, lists, tuples, sets, and dictionaries, including insertion, deletion, searching, and sorting.
- Implement user-defined functions to perform modular programming with parameter passing, return values, and the use of standard Python modules.
- Develop Python programs implementing linear search, binary search, bubble sort, and selection sort for different datasets.
- Write Python programs to create, read, write, append, and manage text files with appropriate exception handling mechanisms.

- Develop Python programs using NumPy to perform array creation, indexing, mathematical operations, and basic matrix manipulation.
- Implement Python programs using Pandas and Matplotlib to read CSV files, perform basic data analysis, and generate graphical visualizations.
- Design and implement a Python-based mini project that integrates computational thinking principles, functions, data structures, file handling, and basic data visualization to solve a real-world problem.

Text Books: 1. MASTERING COMPUTATIONAL THINKING WITH PYTHON: FROM PROBLEM-SOLVING MINDSETS TO REAL-WORLD CODING SOLUTIONS by DR. MALEPATI CHANDRA SEKHAR, MS. RENUKA N D, Clever Fox Publishing

References: 1. COMPUTATIONAL THINKING by PETER J. DENNING, MATTI TEDRE, MIT Press